Minilab 2

1.1

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

3
asdf
benjaminding@NotAnotherMacbook minilab2 % make
make: `output' is up to date.
benjaminding@NotAnotherMacbook minilab2 % make
make: `output' is up to date.
benjaminding@NotAnotherMacbook minilab2 % make
g++ main.cpp class1.cpp class2.cpp -o output
benjaminding@NotAnotherMacbook minilab2 %
```

1.2

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

benjaminding@NotAnotherMacbook minilab2 % touch Makefile
benjaminding@NotAnotherMacbook minilab2 % ls
Makefile        class1.cpp      class1.h        class2.cpp      class2.h        main.cpp
benjaminding@NotAnotherMacbook minilab2 % make
g++ main.cpp class1.cpp class2.cpp -o output
benjaminding@NotAnotherMacbook minilab2 % ./output
3
asdf
benjaminding@NotAnotherMacbook minilab2 %
```

1.3

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

3
asdf
benjaminding@NotAnotherMacbook minilab2 % make
g++ main.cpp class1.cpp class2.cpp -o output
benjaminding@NotAnotherMacbook minilab2 % ./output
Hello World
3
asdf
benjaminding@NotAnotherMacbook minilab2 %
```

The dependencies force the Makefile to check if files have changed and need to be recompiled, because Makefile is an efficiency tool to minimize compilation time it doesn't always try to recompile code it doesn't have to.

2.1

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

make: `output' is up to date.
benjaminding@NotAnotherMacbook minilab2 % make
g++ -c main.cpp -o main.o
g++ -c class1.cpp -o class1.o
g++ -c class2.cpp -o class2.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 % ls
Makefile        class1.cpp      class1.h        class1.o        class2.cpp      class2.h
benjaminding@NotAnotherMacbook minilab2 %
```

2.2

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

g++ -c class1.cpp -o class1.o
g++ -c class2.cpp -o class2.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 % ls
Makefile        class1.cpp      class1.h        class1.o        class2.cpp      class2.h
benjaminding@NotAnotherMacbook minilab2 % make
g++ -c class1.cpp -o class1.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 %
```

2.3

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

g++ -c class1.cpp -o class1.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 % clean
zsh: command not found: clean
benjaminding@NotAnotherMacbook minilab2 % make clean
rm *.o output
benjaminding@NotAnotherMacbook minilab2 % ls
Makefile        class1.cpp      class1.h        class2.cpp      class2.h        main.cpp
benjaminding@NotAnotherMacbook minilab2 %
```

2.4

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

rm: output: No such file or directory
make: *** [clean] Error 1
benjaminding@NotAnotherMacbook minilab2 % make
g++ -c main.cpp -o main.o
benjaminding@NotAnotherMacbook minilab2 % make output
g++ -c class1.cpp -o class1.o
g++ -c class2.cpp -o class2.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 %
```

2.5

```
benjaminding@NotAnotherMacbook minilab2 % make clean
rm *.o output
benjaminding@NotAnotherMacbook minilab2 % g++ -c main.cpp -o main.o
benjaminding@NotAnotherMacbook minilab2 % g++ main.cpp -c -o main.o
benjaminding@NotAnotherMacbook minilab2 % g++ main.cpp -o main.o -c
benjaminding@NotAnotherMacbook minilab2 % g++ -c -o main.o main.cpp
benjaminding@NotAnotherMacbook minilab2 % g++ -o main.o -c main.cpp
benjaminding@NotAnotherMacbook minilab2 % g++ -o main.o main.cpp -c
benjaminding@NotAnotherMacbook minilab2 % g++ -o -c main.o main.cpp
Undefined symbols for architecture arm64:
  "Class1::getX()", referenced from:
      _main in main.o
  "Class1::Class1(int)", referenced from:
      _main in main.o
  "Class2::getS()", referenced from:
      _main in main.o
  "Class2::Class2(std::__1::basic_string<char, std::__1::char_traits<char>, std::__1::allocat
      _main in main.o
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
benjaminding@NotAnotherMacbook minilab2 %
```

3.2

```makefile
# Part 3
output: main.o class1.o class2.o
	g++ $^ -o $@

main.o: main.cpp
	g++ -c $< -o $@

class1.o: class1.cpp
	g++ -c $< -o $@

class2.o: class2.cpp
	g++ -c $< -o $@

clean:
	rm *.o output
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
rm *.o output
rm: output: No such file or directory
make: *** [clean] Error 1
benjaminding@NotAnotherMacbook minilab2 % make
g++ -c main.cpp -o main.o
g++ -c class1.cpp -o class1.o
g++ -c class2.cpp -o class2.o
g++ 	 -o output
clang++: error: no such file or directory: '<86>'
clang++: error: no input files
make: *** [output] Error 1
benjaminding@NotAnotherMacbook minilab2 % make clean
rm *.o output
rm: output: No such file or directory
make: *** [clean] Error 1
benjaminding@NotAnotherMacbook minilab2 % make
g++ -c main.cpp -o main.o
g++ -c class1.cpp -o class1.o
g++ -c class2.cpp -o class2.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 %
```

3.3

- < becomes the leftmost dependency
- @ becomes the target dependency
- ^ becomes all the dependencies

4.2

```
# Part 4
output: main.o class1.o class2.o
	g++ $^ -o $@

%.o: %.cpp
	g++ -c $< -o $@

clean:
	rm *.o output
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
  "Class1::Class1(int)", referenced from:
      _main in main.o
  "Class2::getS()", referenced from:
      _main in main.o
  "Class2::Class2(std::__1::basic_string<char, std::__1::char_traits<char>, std::__1::allocato
      _main in main.o
ld: symbol(s) not found for architecture arm64
clang++: error: linker command failed with exit code 1 (use -v to see invocation)
make: *** [output] Error 1
benjaminding@NotAnotherMacbook minilab2 % make clean
rm *.o output
rm: output: No such file or directory
make: *** [clean] Error 1
benjaminding@NotAnotherMacbook minilab2 % ls
Makefile        class1.cpp      class1.h        class2.cpp      class2.h        main.cpp
benjaminding@NotAnotherMacbook minilab2 % make
g++ -c main.cpp -o main.o
g++ -c class1.cpp -o class1.o
g++ -c class2.cpp -o class2.o
g++ main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 %
```

4.3

- The wildcard operator knows to compile each .cpp file for every .o file declared.

5.1

```
benjaminding@NotAnotherMacbook minilab2 % make
g++ -Wall -c main.cpp -o main.o
g++ -Wall -c class1.cpp -o class1.o
g++ -Wall -c class2.cpp -o class2.o
g++ -Wall main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 %
```

5.2

```
benjaminding@NotAnotherMacbook minilab2 % make
g++ -Wall -std=c++11 -c main.cpp -o main.o
g++ -Wall -std=c++11 -c class1.cpp -o class1.o
g++ -Wall -std=c++11 -c class2.cpp -o class2.o
g++ -Wall -std=c++11 main.o class1.o class2.o -o output
benjaminding@NotAnotherMacbook minilab2 %
```

- FLAGS and COMP are useful for easily identifying certain details in the compilation of code when running, like notifying the user that it is compiled in c++ 11.